

M02 Canonical Lesson Book

14/14 lessons - video first, then solo book. Keep the matching Review closed until a genuine attempt is saved.

DE02-01 - Database, table, row, column and query mental model

VIDEO / VISUAL FIRST

<https://www.youtube.com/watch?v=OT1RErkfLNQ>

Alex The Analyst - SQL Beginner to Advanced | 0:52-11:19 | CURRENT CHECK 2026-09-10

After the assigned visual: close it. The rest of this page is designed to be sufficient for solo learning.

1. Outcome

Install/verify MySQL 8+ and MySQL Workbench, load the course database, and explain the difference between a database, table, row, column, query and result grid.

2. Explain - mental model

A database stores structured objects. A table is a named set of rows and columns. In this lab, sales_orders has one row per order. A SQL query is an instruction sent to the database; the result grid is only the output of that instruction.

3. Demonstrate

```
USE stage02_retail_lab;  
SELECT DATABASE() AS current_database;  
SELECT COUNT(*) AS total_rows FROM sales_orders;
```

4. Guided rebuild

Run the setup script. Confirm the active database is stage02_retail_lab and the table has exactly 30 rows. Open DESCRIBE sales_orders and point to order_id, qty, status and sales_rep.

5. Expected check

Database = stage02_retail_lab; row count = 30; order_id is the primary key; sales_rep can be NULL.

6. Recover

If the database/table is missing, rerun only the setup script and confirm no error occurred before writing learning queries. Do not install WSL for this module; M02 uses MySQL on Windows.

7. Fresh independent transfer

With the video closed, open a new query tab and return order_id, region and product for any 5 rows. In a SQL comment, state the result grain.

8. Validate

Exactly 3 selected columns and 5 rows. Grain statement says one result row represents one order from sales_orders.

9. Explain back

Explain database vs table vs query vs result grid in 45-60 seconds.

10. Mastery evidence

Save your SQL, observed output/count/control, one error or uncertainty, confidence 0-5, and a 30-60 second own-words explanation. If review was needed, mastery requires a fresh changed retry with the review closed.

DE02-02 - SELECT, aliases and LIMIT

VIDEO / VISUAL FIRST

<https://www.youtube.com/watch?v=OT1RErkfLNQ>

Alex The Analyst - SQL Beginner to Advanced | 11:19-21:47 + 47:41-51:10 | CURRENT CHECK 2026-09-10

After the assigned visual: close it. The rest of this page is designed to be sufficient for solo learning.

1. Outcome

Choose output columns, calculate a simple expression, rename output with AS, and limit the number of returned rows.

2. Explain - mental model

SELECT controls the projection: which columns or expressions appear in the result. AS gives an output label. LIMIT reduces the result rows returned after the query logic has run; it does not delete source rows.

3. Demonstrate

```
SELECT order_id, qty, unit_price,  
       qty * unit_price AS gross_sales  
FROM sales_orders  
LIMIT 10;
```

4. Guided rebuild

Return order_id, qty, unit_price and gross_sales for 10 rows. Rename product as item_name in a second query.

5. Expected check

O1001 gross_sales = 1700; O1002 = 1350. First query returns 10 rows and 4 columns.

6. Recover

If the alias is unavailable in the output, check AS spelling and whether the expression has a comma before it. If LIMIT is ignored, confirm it is at the end of the SELECT statement.

7. Fresh independent transfer

Return order_id, order_date and product for every row, then create a separate query returning 7 rows with product renamed item_name.

8. Validate

First query returns 30 rows / 3 columns. Second returns 7 rows and has a column named item_name.

9. Explain back

Explain why SELECT and LIMIT change the result but not the stored table.

10. Mastery evidence

Save your SQL, observed output/count/control, one error or uncertainty, confidence 0-5, and a 30-60 second own-words explanation. If review was needed, mastery requires a fresh changed retry with the review closed.

DE02-03 - WHERE and comparison operators

VIDEO / VISUAL FIRST

<https://www.youtube.com/watch?v=OT1RErkfLNQ>

Alex The Analyst - SQL Beginner to Advanced | 21:47-33:54 | CURRENT CHECK 2026-09-10

After the assigned visual: close it. The rest of this page is designed to be sufficient for solo learning.

1. Outcome

Translate a single business condition into WHERE filters using =, <>, >, >=, <, <=, BETWEEN and IN.

2. Explain - mental model

WHERE decides which source rows are eligible before grouping. Every returned row must satisfy the condition. A filter should be readable as the business rule you were given.

3. Demonstrate

```
SELECT order_id, region, product, qty, status
FROM sales_orders
WHERE status = 'Completed' AND qty >= 2;
```

4. Guided rebuild

Run the Completed AND qty >= 2 filter and a separate COUNT(*) using the identical WHERE clause.

5. Expected check

22 rows; no Pending or Cancelled rows survive.

6. Recover

If the count differs, read the English rule and test each predicate separately. Do not edit several clauses at once.

7. Fresh independent transfer

Return orders dated 2026-07-10 through 2026-07-12 inclusive, then return only West orders with unit_price >= 850.

8. Validate

Date-range result = 6 rows. West/high-price result contains only West and unit_price >= 850.

9. Explain back

Explain WHERE as an eligibility rule, not just syntax.

10. Mastery evidence

Save your SQL, observed output/count/control, one error or uncertainty, confidence 0-5, and a 30-60 second own-words explanation. If review was needed, mastery requires a fresh changed retry with the review closed.

DE02-04 - AND, OR, NOT and parentheses

VIDEO / VISUAL FIRST

<https://www.youtube.com/watch?v=OT1RErkfLNQ>

Alex The Analyst - SQL Beginner to Advanced | 21:47-33:54 | CURRENT CHECK 2026-09-10

After the assigned visual: close it. The rest of this page is designed to be sufficient for solo learning.

1. Outcome

Combine conditions safely and use parentheses so Boolean logic matches the business request.

2. Explain - mental model

AND requires both conditions. OR allows either. NOT reverses a condition. Parentheses make intended grouping explicit and prevent operator precedence from silently changing eligibility.

3. Demonstrate

```
SELECT order_id, region, status
FROM sales_orders
WHERE status='Completed'
AND (region='East' OR region='North');
```

4. Guided rebuild

Return Completed orders from East or North with explicit parentheses. Count the rows separately.

5. Expected check

13 rows. Regions are only East/North and every status is Completed.

6. Recover

If a North Pending row appears, inspect the parentheses. Translate the WHERE clause back into English before rerunning.

7. Fresh independent transfer

Return non-Cancelled orders from East or North. Then write a logically equivalent version using NOT status = 'Cancelled'.

8. Validate

Both independent queries return the same order_ids. Save a set/count comparison.

9. Explain back

Explain why A AND (B OR C) is different from (A AND B) OR C.

10. Mastery evidence

Save your SQL, observed output/count/control, one error or uncertainty, confidence 0-5, and a 30-60 second own-words explanation. If review was needed, mastery requires a fresh changed retry with the review closed.

DE02-05 - ORDER BY, DISTINCT and result inspection

VIDEO / VISUAL FIRST

<https://www.youtube.com/watch?v=OT1RErkfLNQ>

Alex The Analyst - SQL Beginner to Advanced | 11:19-21:47 + 47:41-51:10 | CURRENT CHECK 2026-09-10

After the assigned visual: close it. The rest of this page is designed to be sufficient for solo learning.

1. Outcome

Remove repeated result values with DISTINCT, sort intentionally, and use top-N inspection without confusing sorting with filtering.

2. Explain - mental model

DISTINCT removes duplicate combinations from the query result. ORDER BY changes sequence. LIMIT keeps only the first N rows after sorting. None of these delete source data.

3. Demonstrate

```
SELECT DISTINCT region FROM sales_orders ORDER BY region;

SELECT order_id, product, qty*unit_price AS gross_sales
FROM sales_orders
ORDER BY gross_sales DESC
LIMIT 5;
```

4. Guided rebuild

List unique regions alphabetically. In a separate query return the five highest gross-sales orders.

5. Expected check

Regions = East, North, South, West. Highest gross order = O1020 at 3600.

6. Recover

For a top-N question, sort DESC before LIMIT. If DISTINCT returns more values than expected, inspect all selected columns: DISTINCT applies to the whole selected combination.

7. Fresh independent transfer

Return distinct payment methods alphabetically. Then return the three largest-quantity orders with a deterministic tie-break on order_id.

8. Validate

Exactly Card, Cash, UPI. Top qty begins O1021=10, O1026=9, O1007=8.

9. Explain back

Explain DISTINCT on one column versus DISTINCT on two columns.

10. Mastery evidence

Save your SQL, observed output/count/control, one error or uncertainty, confidence 0-5, and a 30-60 second own-words explanation. If review was needed, mastery requires a fresh changed retry with the review closed.

DE02-06 - NULL, IS NULL and COALESCE

VIDEO / VISUAL FIRST

https://www.youtube.com/watch?v=ol9_NnC9-cc

Luke Barousse - Data Engineer Bootcamp (2026) | Functions: Date, Text & NULL - focus NULL/COALESCE | RC8 CURRENT MAP; FINAL BOOK FALLBACK INCLUDED

After the assigned visual: close it. The rest of this page is designed to be sufficient for solo learning.

1. Outcome

Find missing values correctly, explain COUNT(*) versus COUNT(column), and use COALESCE only when a display/default rule is justified.

2. Explain - mental model

NULL means no value is present; it is not zero and not empty text. Use IS NULL / IS NOT NULL rather than = NULL. COUNT(*) counts rows; COUNT(column) counts non-NULL values in that column. COALESCE returns the first non-NULL expression.

3. Demonstrate

```
SELECT COUNT(*) AS rows_all, COUNT(sales_rep) AS rows_with_rep
FROM sales_orders;

SELECT order_id, COALESCE(sales_rep, 'Unassigned') AS rep_display
FROM sales_orders
WHERE sales_rep IS NULL;
```

4. Guided rebuild

Compare COUNT(*) with COUNT(sales_rep), find the NULL row, and display Unassigned with COALESCE without updating the table.

5. Expected check

30 total rows; 29 non-NULL sales_rep; O1030 is the one NULL sales_rep row.

6. Recover

If = NULL returns nothing, replace it with IS NULL. Do not use COALESCE to pretend missing source data is known; here it is a presentation label only.

7. Fresh independent transfer

Return all rows with sales_rep IS NOT NULL and prove their count. Then show every order with a rep_label that uses Unassigned only for display.

8. Validate

IS NOT NULL count = 29. Source sales_rep for O1030 remains NULL.

9. Explain back

Explain NULL, IS NULL, COUNT(column), and COALESCE with O1030.

10. Mastery evidence

Save your SQL, observed output/count/control, one error or uncertainty, confidence 0-5, and a 30-60 second own-words explanation. If review was needed, mastery requires a fresh changed retry with the review closed.

DE02-07 - Arithmetic expressions and safe calculations

VIDEO / VISUAL FIRST

https://www.youtube.com/watch?v=ol9_NnC9-cc

Luke Barousse - Data Engineer Bootcamp (2026) | Operators & Functions - arithmetic focus | RC8 CURRENT MAP; FINAL BOOK
FALLBACK INCLUDED

After the assigned visual: close it. The rest of this page is designed to be sufficient for solo learning.

1. Outcome

Build auditable gross/net calculations from source columns and validate them manually.

2. Explain - mental model

A calculated result should show its business formula. $\text{gross_sales} = \text{qty} * \text{unit_price}$. $\text{net_sales} = \text{gross_sales} * (1 - \text{discount_pct})$. A discount_pct of 0.05 means 5%, not 5 currency units.

3. Demonstrate

```
SELECT order_id, qty, unit_price, discount_pct,  
       qty*unit_price AS gross_sales,  
       qty*unit_price*(1-discount_pct) AS net_sales  
FROM sales_orders;
```

4. Guided rebuild

Calculate gross_sales and net_sales for all orders. Manually verify O1002.

5. Expected check

O1002 gross = 1350; net = 1282.50.

6. Recover

If net is larger than gross for a positive discount, inspect the sign. If decimals look strange, round for presentation only after the underlying calculation is correct.

7. Fresh independent transfer

Calculate $\text{discount_value} = \text{gross_sales} - \text{net_sales}$ and validate two orders manually.

8. Validate

For every row with $\text{discount_pct} > 0$, $\text{discount_value} > 0$ and $\text{net_sales} \leq \text{gross_sales}$. Save two manual checks.

9. Explain back

Explain the difference between stored source fields and derived query expressions.

10. Mastery evidence

Save your SQL, observed output/count/control, one error or uncertainty, confidence 0-5, and a 30-60 second own-words explanation. If review was needed, mastery requires a fresh changed retry with the review closed.

DE02-08 - Aggregate functions

VIDEO / VISUAL FIRST

<https://www.youtube.com/watch?v=OT1RErkfLNQ>

Alex The Analyst - SQL Beginner to Advanced | 33:54-44:18 | CURRENT CHECK 2026-09-10

After the assigned visual: close it. The rest of this page is designed to be sufficient for solo learning.

1. Outcome

Summarize eligible rows with COUNT, SUM, AVG, MIN and MAX and explain what NULL does to common aggregates.

2. Explain - mental model

Without GROUP BY, an aggregate summarizes the filtered input into one result row. COUNT(*) counts rows, while COUNT(column) ignores NULL values. SUM/AVG/MIN/MAX operate on the values that survive filtering.

3. Demonstrate

```
SELECT COUNT(*) AS order_rows, SUM(qty) AS total_qty,  
       AVG(unit_price) AS avg_unit_price,  
       MIN(unit_price) AS min_price, MAX(unit_price) AS max_price,  
       SUM(qty*unit_price) AS gross_sales  
FROM sales_orders;
```

4. Guided rebuild

Return row count, total qty, average price, min/max price and gross sales for all rows.

5. Expected check

30 rows; total qty 105; average unit_price about 666.33; gross sales 50000.

6. Recover

If an aggregate is unexpectedly high/low, first inspect the eligible row count and whether the filter matches the business rule.

7. Fresh independent transfer

For Completed orders only, return order count, total qty and total net_sales in one summary row.

8. Validate

Completed order count = 27; total qty = 100; total net_sales = 42984.50.

9. Explain back

Explain why a one-row aggregate has a different result grain from the source table.

10. Mastery evidence

Save your SQL, observed output/count/control, one error or uncertainty, confidence 0-5, and a 30-60 second own-words explanation. If review was needed, mastery requires a fresh changed retry with the review closed.

DE02-09 - GROUP BY and grouped grain

VIDEO / VISUAL FIRST

<https://www.youtube.com/watch?v=OT1RErkfLNQ>

Alex The Analyst - SQL Beginner to Advanced | 33:54-44:18 | CURRENT CHECK 2026-09-10

After the assigned visual: close it. The rest of this page is designed to be sufficient for solo learning.

1. Outcome

Create one summary row per group, state output grain first, and reconcile grouped totals to an overall control.

2. Explain - mental model

GROUP BY changes result grain. If you GROUP BY region, one output row represents one region. Selected non-aggregate columns must belong to the grouping grain.

3. Demonstrate

```
SELECT region, COUNT(*) AS orders, SUM(qty) AS total_qty,
SUM(qty*unit_price) AS gross_sales
FROM sales_orders
WHERE status='Completed'
GROUP BY region
ORDER BY region;
```

4. Guided rebuild

Build Completed order count and gross sales by region.

5. Expected check

East 8 / 12800; North 5 / 9310; South 7 / 12010; West 7 / 10930.

6. Recover

If grouped totals do not reconcile, run the same measure with the same WHERE filter without GROUP BY. Do not trust plausible-looking group rows.

7. Fresh independent transfer

Build Completed order count, qty and net_sales by payment_method and state the result grain.

8. Validate

3 result rows; grouped net total reconciles to 42984.50.

9. Explain back

Explain how GROUP BY changes grain from one row/order to one row/group.

10. Mastery evidence

Save your SQL, observed output/count/control, one error or uncertainty, confidence 0-5, and a 30-60 second own-words explanation. If review was needed, mastery requires a fresh changed retry with the review closed.

DE02-10 - HAVING after aggregation

VIDEO / VISUAL FIRST

<https://www.youtube.com/watch?v=OT1RErkfLNQ>

Alex The Analyst - SQL Beginner to Advanced | 44:18-47:41 | CURRENT CHECK 2026-09-10

After the assigned visual: close it. The rest of this page is designed to be sufficient for solo learning.

1. Outcome

Use WHERE for source-row eligibility and HAVING for conditions on grouped/aggregate results.

2. Explain - mental model

WHERE runs before aggregation and filters source rows. HAVING filters groups after GROUP BY. A condition such as status=Completed belongs in WHERE; a condition such as SUM(qty)>=25 belongs in HAVING.

3. Demonstrate

```
SELECT region, SUM(qty) AS total_qty
FROM sales_orders
GROUP BY region
HAVING SUM(qty) >= 25
ORDER BY total_qty DESC;
```

4. Guided rebuild

Group all rows by region and keep groups with SUM(qty) >= 25.

5. Expected check

East 29, North 27, West 25; South is excluded.

6. Recover

If you put SUM(qty) in WHERE, move the aggregate condition to HAVING. If business eligibility is wrong, inspect WHERE separately.

7. Fresh independent transfer

For Completed orders only, return payment methods whose total qty is at least 25.

8. Validate

Cash 39 and UPI 39 qualify; Card 22 does not.

9. Explain back

Say when WHERE happens and when HAVING happens, using one example of each.

10. Mastery evidence

Save your SQL, observed output/count/control, one error or uncertainty, confidence 0-5, and a 30-60 second own-words explanation. If review was needed, mastery requires a fresh changed retry with the review closed.

DE02-11 - CASE for business categories

VIDEO / VISUAL FIRST

<https://www.youtube.com/watch?v=OT1RErkfLNQ>

Alex The Analyst - SQL Beginner to Advanced | 1:26:39-1:35:16 | CURRENT CHECK 2026-09-10

After the assigned visual: close it. The rest of this page is designed to be sufficient for solo learning.

1. Outcome

Translate business rules into ordered CASE branches and validate bucket coverage.

2. Explain - mental model

CASE tests WHEN branches from top to bottom and returns the first matching result. For overlapping thresholds, put the highest/more restrictive rule first.

3. Demonstrate

```
SELECT order_id, qty*unit_price AS gross_sales,  
CASE WHEN qty*unit_price >= 2500 THEN 'High'  
WHEN qty*unit_price >= 1500 THEN 'Medium'  
ELSE 'Small' END AS order_size  
FROM sales_orders;
```

4. Guided rebuild

Bucket all orders into High >=2500, Medium >=1500, otherwise Small, then count each bucket.

5. Expected check

High 3; Medium 14; Small 13.

6. Recover

If High is empty, inspect branch order. If counts do not sum to 30, inspect missing ELSE or filter logic.

7. Fresh independent transfer

Create fulfillment_group: Completed -> Closed; Pending -> Open; otherwise Other. Count each group.

8. Validate

Counts across the three groups sum to 30 and source status values remain unchanged.

9. Explain back

Explain why CASE is derived result logic and not an UPDATE.

10. Mastery evidence

Save your SQL, observed output/count/control, one error or uncertainty, confidence 0-5, and a 30-60 second own-words explanation. If review was needed, mastery requires a fresh changed retry with the review closed.

DE02-12 - String and date basics

VIDEO / VISUAL FIRST

<https://www.youtube.com/watch?v=OT1RErkfLNQ>

Alex The Analyst - SQL Beginner to Advanced | 1:15:10-1:26:39 | CURRENT CHECK 2026-09-10

After the assigned visual: close it. The rest of this page is designed to be sufficient for solo learning.

1. Outcome

Use basic text cleanup/display functions and extract date parts while preserving source data.

2. Explain - mental model

TRIM removes surrounding whitespace from a result expression, UPPER changes display casing, and YEAR/MONTH extract calendar components in MySQL. These query expressions do not update source values.

3. Demonstrate

```
SELECT order_id, UPPER(TRIM(region)) AS region_clean,  
       YEAR(order_date) AS order_year, MONTH(order_date) AS order_month  
FROM sales_orders  
WHERE order_date BETWEEN '2026-07-10' AND '2026-07-12';
```

4. Guided rebuild

Run the date-range/text query for July 10 through July 12 inclusive.

5. Expected check

6 rows; every order_year = 2026 and order_month = 7.

6. Recover

If dates are missing, check endpoints and data type. If text cleanup changes the business meaning, do not treat it as a safe standardization without a rule.

7. Fresh independent transfer

Return product labels as UPPER(product) and extract day-of-month for orders on July 13-15. Keep original product beside the transformed value.

8. Validate

Only July 13, 14 and 15 rows appear; transformed output is traceable to original text.

9. Explain back

Explain query-time transformation versus changing stored source data.

10. Mastery evidence

Save your SQL, observed output/count/control, one error or uncertainty, confidence 0-5, and a 30-60 second own-words explanation. If review was needed, mastery requires a fresh changed retry with the review closed.

DE02-13 - Simple subqueries and IN/EXISTS awareness

VIDEO / VISUAL FIRST

<https://www.youtube.com/watch?v=OT1RErkfLNQ>

Alex The Analyst - SQL Beginner to Advanced | 1:35:16-1:45:58 | CURRENT CHECK 2026-09-10

After the assigned visual: close it. The rest of this page is designed to be sufficient for solo learning.

1. Outcome

Use a simple scalar subquery and recognize IN/EXISTS as ways to filter based on another query result.

2. Explain - mental model

A subquery is a query nested inside another query. A scalar subquery returns one value, such as an average. IN compares a value against a set returned by a subquery. EXISTS asks whether a matching row exists. Keep the first exercises simple; complex correlated logic belongs later.

3. Demonstrate

```
SELECT order_id, qty*unit_price*(1-discount_pct) AS net_sales
FROM sales_orders
WHERE status='Completed'
AND qty*unit_price*(1-discount_pct) > (
  SELECT AVG(qty*unit_price*(1-discount_pct))
  FROM sales_orders
  WHERE status='Completed'
)
ORDER BY net_sales DESC;
```

4. Guided rebuild

Find Completed orders whose net_sales is above the average Completed net_sales. Run the inner average query alone first.

5. Expected check

Average Completed net_sales is about 1592.02; 13 orders are above it.

6. Recover

If the outer result is wrong, run the inner query independently and verify it returns one value. Keep the same Completed eligibility in both places.

7. Fresh independent transfer

Use IN with a subquery to return orders whose region belongs to a region whose all-status SUM(qty) is at least 25. State what the inner query returns.

8. Validate

The inner set is East, North, West; no South order appears in the outer result.

9. Explain back

Explain scalar subquery versus IN in simple English; give one reason to validate the inner query separately.

10. Mastery evidence

Save your SQL, observed output/count/control, one error or uncertainty, confidence 0-5, and a 30-60 second own-words explanation. If review was needed, mastery requires a fresh changed retry with the review closed.

DE02-14 - Debugging and validation workflow

VIDEO / VISUAL FIRST

<https://www.metabase.com/learn/sql/debugging-sql/sql-logic>

Metabase Learn - Debugging SQL query logic + course-owned validation lab | Current short debugging guides; no forced weak video |
CURRENT WEB CHECK 2026-09-10

After the assigned visual: close it. The rest of this page is designed to be sufficient for solo learning.

1. Outcome

Debug syntax/logic problems systematically and prove a result with independent counts, grain checks and reconciliations rather than trusting a green Run button.

2. Explain - mental model

Debugging has two branches: the query fails to run, or it runs but the answer is wrong. For syntax errors, read the exact error and reduce the query. For wrong results, verify source grain, filters, row counts and aggregates. Change one suspected cause at a time.

3. Demonstrate

```
-- Wrong business rule: Pending rows are accidentally included
SELECT region, SUM(qty*unit_price) AS gross_sales
FROM sales_orders
WHERE status <> 'Cancelled'
GROUP BY region;

-- Control
SELECT COUNT(*) FROM sales_orders WHERE status='Completed';
```

4. Guided rebuild

Run the wrong-business-rule query, compare its eligibility count with Completed-only count, repair the filter, then reconcile grouped gross_sales to a separate Completed overall total.

5. Expected check

Completed-only count = 27. Completed grouped gross total reconciles to 45050. A query that includes Pending is not equivalent even if the numbers look plausible.

6. Recover

Do not randomly edit SQL. Record expected result, actual result, exact error/wrong signal, one hypothesis, one change and rerun evidence. Preserve the failed attempt.

7. Fresh independent transfer

Take a deliberately wrong query that uses OR without parentheses, diagnose why a Pending row leaks into the output, repair it, and save before/after row counts plus the corrected English rule.

8. Validate

The corrected query has the intended status and regions only. A separate COUNT/control query supports the result.

9. Explain back

Explain the difference between a syntax bug, a logic bug and a validation check. Give one example of each.

10. Mastery evidence

Save your SQL, observed output/count/control, one error or uncertainty, confidence 0-5, and a 30-60 second own-words explanation. If review was needed, mastery requires a fresh changed retry with the review closed.